

Technical Implementation Report

Frontend, Node.js backend, Supabase data model, security hardening, and workflow enhancements completed through 17 September 2026.

Repositories	latestMidori (React/Vite) and MidoriBackend (Node.js/Express)
Primary stack	React, TypeScript, Express, Supabase, Cloudflare R2, Nodemailer
Delivery status	Implementation complete; Supabase migration must be applied before deployment

1. Delivery Summary

The project was enhanced without rotating or replacing existing secrets. The work focused on correctness, access control, client-facing workflows, and operational polish while preserving the established visual system and application structure.

Area	Delivered capability
Client lifecycle	Generated credentials, secure password storage, protected reset flow, structured onboarding data
Project workflows	Role-aware contract signing, access checks, project-data validation
Packages	Selected package and add-on details sent by email via Nodemailer
Vendor media	Consent persistence, vendor registry, vendor-linked files, grouped media access and downloads
Quality	Frontend production build and backend syntax validation completed

Frontend implementation

The React application was extended in the existing UI language instead of being redesigned.

- Client onboarding and project views now consume structured client and event data consistently.
- Client credentials are masked by default in the UI; privileged password-reset actions require the authenticated administrator's password before a new temporary credential is returned.
- Contract controls are presentation-gated for non-clients, matching server-side enforcement.
- The package builder sends the selected package, features, add-ons, pricing, timeline, budget and notes with the enquiry.
- Upload Assets supports vendor-sharing selection, a required vendor association, admin-only inline vendor creation, and clear non-admin guidance.
- Vendor Media Access is grouped by vendor, includes intentional loading/error/empty states, video and unavailable-preview fallbacks, single-file download and a rate-limited download-all queue.
- Vendor consent dialog is rendered only for clients that have not accepted; it includes focus management, Escape handling, ARIA dialog metadata and restoration of focus to the invoking control.

Backend implementation

The Node.js/Express API now encapsulates the business rules that the UI depends upon.

- JWT authentication and authorization middleware were strengthened with client, file and project-step ownership checks.
- Client creation generates a temporary credential with cryptographic randomness; only a bcrypt hash is persisted. Legacy password compatibility and transparent hash upgrade were included.
- A protected password-reset endpoint verifies the administrator's own password before generating a replacement temporary client password.
- Contract-signing routes now require client identity and verify that the client can sign only their own contract.
- File upload persists vendor-share state and validates that a chosen vendor belongs to the same client project.
- Vendor endpoints list and create vendors, persist consent, and retrieve signed R2 media URLs grouped by vendor. Client API access is also blocked until consent is saved.
- Nodemailer package-enquiry emails render the selected commercial configuration rather than a generic message.

2. Vendor Media Architecture

Vendor media is implemented as a project-scoped, consent-gated sharing model. It avoids exposing unrelated project media and enforces file-to-vendor association on both the application and database boundaries.

Step	System behavior
1. Consent	A client entering Vendor Media Access sees the consent dialog only when <code>clients.vendor_media_consent</code> is false. Acceptance updates the client record with a timestamp.
2. Vendor creation	Administrators can create a vendor within the upload workflow. Non-admin users receive a clear authorization message.
3. File upload	A vendor-shared upload must include <code>vendor_id</code> . The backend verifies ownership and stores <code>is_vendor_shared</code> plus <code>vendor_id</code> .
4. Media retrieval	The API selects only <code>is_vendor_shared = true</code> files for the current client, joins them logically to vendors, then removes zero-file vendor groups.
5. Access	Signed R2 URLs are short-lived. Direct client API access is rejected when consent is absent.

Supabase schema additions

The migration file is located at `MidoriBackend/supabase/migrations/20260916_project_data_hardening.sql`.

- `clients.vendor_media_consent` BOOLEAN NOT NULL DEFAULT false
- `clients.vendor_media_consent_at` TIMESTAMPTZ
- vendors table: `vendor_id`, `client_id`, `vendor_name`, `vendor_type`, `contact_email`, `contact_phone`, `created_by`, `created_at` and `updated_at`
- `files.vendor_id` UUID foreign key to `vendors.vendor_id`, using ON DELETE SET NULL
- Indexes on `vendors.client_id` and `files.vendor_id`
- A files trigger that requires `vendor_id` for shared files, clears `vendor_id` for non-shared files, and prevents cross-client vendor linkage.

Deployment prerequisite

Apply the supplied Supabase migration before deploying the backend/frontend release. Historical files with `is_vendor_shared = true` and no `vendor_id` are intentionally left unchanged because their vendor cannot be inferred safely; they should be assigned during a controlled data-cleanup pass.

3. Security and Reliability

Security controls

- Passwords are stored with bcrypt hashes; plaintext passwords are never retained in the database.
- Credential reset requires a verified administrator password and returns the temporary credential only at reset time.
- Authentication middleware recognizes bearer/cookie identity, enforces resource ownership, and separates client-only actions from internal team actions.
- Vendor-media consent is enforced in the API, not merely hidden in the frontend.
- R2 media delivery uses short-lived signed URLs, and file download checks remain behind authenticated routes.
- HTTP hardening and rate limiting dependencies were added to the server runtime.

Data integrity

- The file `vendor_id` is nullable for non-vendor assets and mandatory for new vendor-shared assignments.
- Database trigger validation prevents a file from referencing a vendor owned by a different client.
- Vendor deletion is non-destructive to files: files retain their records and set `vendor_id` to NULL through the foreign-key delete policy.
- Client-level indexes were added for frequently queried project data to reduce lookup overhead.

Verification performed

- Vite production build completed successfully: 2,391 modules transformed and production assets generated.
- Node.js syntax checks passed for the Express entry point, vendor controller, upload controller, project controller and vendor routes.
- Git diff whitespace validation passed for backend changes.
- The final design review drove added empty/error/fallback states and modal accessibility refinements. Browser screenshot automation was unavailable in the local environment, so visual verification should be repeated after deployment.

4. Operational Notes

What was deliberately not changed

- No environment secrets, API keys or credentials were rotated or altered.
- Existing assets were preserved; the production build confirms the previously referenced image assets resolve.
- Existing core UI patterns and application behavior were retained; additions were scoped to the requested flows.

Recommended release sequence

- Back up the Supabase schema and apply the migration in the SQL Editor or migration runner.
- Audit historical vendor-shared files with `vendor_id IS NULL` and assign a vendor where known.
- Deploy MidoriBackend and latestMidori together so frontend calls align with the new vendor endpoints.
- Perform a role-based smoke test: admin creates vendor/uploads media; client accepts consent/views/downloads; non-admin attempts vendor creation; a client attempts another project's media.
- Confirm outbound Nodemailer delivery in the target environment using the configured mail provider credentials.

Prepared for Midori Media. This report documents implemented application changes and the corresponding deployment dependencies as of 17 September 2026.